

# 00 树状数组

## 树状数组板子

### 1. 单点加, 区间查询

适用背景:

维护一个数组  $a$ , 支持:

1  $x \ v \quad a[x] += v$

2  $l \ r \quad$  查询  $\text{sum}[l, r]$

这是最基础的树状数组。

常见应用:

动态维护前缀和

单点修改, 区间求和

逆序对统计

扫描线辅助统计

频率数组维护

```
struct BIT{ // 单点加, 区间查询 (BIT_Point_Add____Range_Query)
    int n;
    vector<ll> tr;
    BIT(int _n = 0): n(_n), tr(_n + 2) {} // 初始化
    // void init(int _n){ // 全局 bit 初始化才需要
    //     n = _n;
    //     tr.assign(n + 2, 0);
    // }

    void add(int x, ll v){ // 单点加
        for(; x <= n; x += x & -x) tr[x] += v;
    }

    ll query(int x){ // 单点查询: 查前缀和 a[1, x]
        ll ans = 0;
        for(; x >= 1; x -= x & -x) ans += tr[x];
        return ans;
    }

    ll query(int l, int r){ // 区间查询: 即区间和 query[x, y]
        return query(r) - query(l - 1);
    }
};

// BIT bit; // 全局 bit

void work(){
    int n, q;
    cin >> n >> q;
    BIT bit(n);
```

```

        for(int i = 1; i <= n; ++i){
            ll x;
            cin >> x;
            bit.add(i, x);
        }
        while(q--){
            int op;
            cin >> op;
            if(op == 1){
                int x;
                ll v;
                cin >> x >> v;
                bit.add(x, v);
            }else{
                int l, r;
                cin >> l >> r;
                cout << bit.query(l, r) << '\n';
            }
        }
    }
}

```

## 2.区间加, 单点查询

### 适用背景:

维护一个数组  $a$ , 支持:

1 l r v  $a[l, r] += v$

2 x 查询  $a[x]$

核心思想是维护差分数组  $d$ 。

如果:

$a[i] = d[1] + d[2] + \dots + d[i]$

那么区间加:  $a[l, r] += v$

等价于:  $d[l] += v$   $d[r + 1] -= v$

### 常见应用:

区间整体加

最后查询某些点的值

离线区间覆盖统计

差分思想 + BIT 动态化

```

struct BIT{ // 区间加, 单点查询 (BIT_Range_Add___Point_Query)
    int n;
    vector<ll> tr;
    BIT(int _n = 0): n(_n), tr(_n + 2) {} // 初始化
    void add(int x, ll v){ // 单点加
        for(; x <= n; x += x & -x) tr[x] += v;
    }
}

```

```

    }

    void add(int l, int r, ll v){ // 区间加
        add(l, v);
        add(r + 1, -v);
    }

    ll query(int x){ // 单点查询：查前缀和 a[1, x]
        ll ans = 0;
        for(; x >= 1; x -= x & -x) ans += tr[x];
        return ans;
    }
};

void work(){
    int n, q;
    cin >> n >> q;
    BIT bit(n);
    for(int i = 1; i <= n; ++i){
        ll x;
        cin >> x;
        bit.add(i, i, x);
    }

    while(q--){
        int op;
        cin >> op;
        if(op == 1){
            int l, r;
            ll v;
            cin >> l >> r >> v;
            bit.add(l, r, v);
        }else{
            int x;
            cin >> x;
            cout << bit.query(x) << '\n';
        }
    }
}

```

### 3. 区间加, 区间查询

适用背景：

维护一个数组  $a$ ，支持：

1 |  $r \ v \ a[l, r] += v$

2 |  $r \ \text{查询 } \text{sum}[l, r]$

这个功能比前两个都强，用两个 BIT 维护。

核心公式：

设差分数组为 d。

$$a[1] + a[2] + \dots + a[x] = \sum d[i] (x - i + 1) = (x + 1) \sum d[i] - \sum d[i] i$$

所以维护两个树状数组：s1 维护  $d[i]$  s2 维护  $d[i] i$

前缀和： $\text{sum}(x) = (x + 1) * s1.\text{query}(x) - s2.\text{query}(x)$

```
struct BIT{ // 区间加， 区间查询 (BIT_Range_Add___Range_Query)
    int n;
    vector<ll> tr;
    BIT(int _n = 0): n(_n), tr(_n + 2) {}
    void add(int x, ll v){ // 单点加
        for(; x <= n; x += x & -x) tr[x] += v;
    }
    ll query(int x){ // 单点查询：查前缀和 a[1, x]
        ll ans = 0;
        for(; x >= 1; x -= x & -x) ans += tr[x];
        return ans;
    }
};

struct RBIT{
    int n;
    BIT s1, s2; // 用两个树状数组维护 "区间加、区间查" 功能
    RBIT(int _n): n(_n), s1(_n), s2(_n) {}
    void add(int x, ll v){
        s1.add(x, v), s2.add(x, v * x); // 两次单点加
    }
    void add(int l, int r, ll v){
        add(l, v), add(r + 1, -v); // 两次单点查询
    }
    ll query(int x){
        return (x + 1) * s1.query(x) - s2.query(x);
    }
    ll query(int l, int r){
        return query(r) - query(l - 1);
    }
};

void work(){
    int n, q;
    cin >> n >> q;
    RBIT bit(n);
    for(int i = 1; i <= n; ++i){
        ll x;
        cin >> x;
        bit.add(i, i, x);
    }
    while(q--){
        int op;
        cin >> op;
```

```

        if(op == 1){
            int l, r;
            ll v;
            cin >> l >> r >> v;
            bit.add(l, r, v);
        }else{
            int l, r;
            cin >> l >> r;
            cout << bit.query(l, r) << '\n';
        }
    }
}

```

## 4. 权值树状数组

### 适用背景：

树状数组维护的不是原数组，而是“值域频率”。

例如值域是  $1 \sim n$ ，支持：

插入一个数  $x$

删除一个数  $x$

查询  $\leq x$  的数有多少个

查询当前第  $k$  小

### 常见应用：

动态第  $k$  小

逆序对

排名查询

离散化后维护频率

多重集合替代品

```

struct BIT{ // 权值树状数组
    int n;
    vector<int> tr;
    BIT(int _n): n(_n), tr(_n + 2) {}
    void add(int x, int v){ // 单点加
        for(; x <= n; x += x & -x){
            tr[x] += v;
        }
    }
    int query(int x){ // 单点查询：查前缀和 a[1, x]
        int ans = 0;
        for(; x >= 1; x -= x & -x){
            ans += tr[x];
        }
        return ans;
    }
}

```

```

    }

    int kth(int k){
        int x = 0;
        for(int i = 1 << __lg(n); i; i >>= 1){
            if(x + i <= n && tr[x + i] < k){
                x += i;
                k -= tr[x];
            }
        }
        return x + 1;
    }
};

void work(){
    int n, q;
    cin >> n >> q;
    BIT bit(n);
    while(q--){
        int op, x;
        cin >> op >> x;
        if(op == 1){
            bit.add(x, 1);          // 插入 x
        }else if(op == 2){
            bit.add(x, -1);         // 删除 x
        }else if(op == 3){
            cout << bit.query(x) << '\n'; // <= x 的数量
        }else{
            cout << bit.kth(x) << '\n';  // 第 x 小
        }
    }
}
}

```

## 5. 二维树状数组

### 适用背景：

维护一个矩阵 a，支持：

单点加, 矩阵查询

### 常见应用：

二维前缀和动态版

矩阵单点修改，矩形求和

平面点统计

离线扫描二维偏序

```

struct BIT2{
    int n, m;

```

```

vector<vector<ll>> tr;

BIT2(int _n, int _m): n(_n), m(_m), tr(n + 2, vector<ll>(m + 2)) {}

void add(int x, int y, ll v){
    for(int i = x; i <= n; i += i & -i){
        for(int j = y; j <= m; j += j & -j){
            tr[i][j] += v;
        }
    }
}

ll query(int x, int y){
    ll ans = 0;
    for(int i = x; i >= 1; i -= i & -i){
        for(int j = y; j >= 1; j -= j & -j){
            ans += tr[i][j];
        }
    }
    return ans;
}

ll query(int x1, int y1, int x2, int y2){
    return query(x2, y2) - query(x1 - 1, y2) - query(x2, y1 - 1) +
    query(x1 - 1, y1 - 1);
}

};

void work(){
    int n, m, q;
    cin >> n >> m >> q;
    BIT2 bit(n, m);
    for(int i = 1; i <= n; ++i){
        for(int j = 1; j <= m; ++j){
            ll x;
            cin >> x;
            bit.add(i, j, x);
        }
    }

    while(q--){
        int op;
        cin >> op;
        if(op == 1){
            int x, y;
            ll v;
            cin >> x >> y >> v;
            bit.add(x, y, v);
        }else{
            int x1, y1, x2, y2;
            cin >> x1 >> y1 >> x2 >> y2;
            cout << bit.query(x1, y1, x2, y2) << '\n';
        }
    }
}

```

```
}  
}
```

## 6. 前缀最大值树状数组

### 适用背景：

普通 BIT 维护的是 "和"，但如果操作是单调更新，也可以维护最大值。

### 常见应用：

LIS 优化

DP 转移最大值

权值压缩后求前缀最优

偏序 DP

注意：这种 BIT 一般不支持普通删除，也不支持随便改小。

```
struct BIT{  
    int n;  
    vector<ll> tr;  
    BIT(int _n = 0): n(_n), tr(_n + 2, -INF) {}  
    void update(int x, ll v){  
        for(; x <= n; x += x & -x) tr[x] = max(tr[x], v);  
    }  
    ll query(int x){  
        ll ans = -INF;  
        for(; x >= 1; x -= x & -x) ans = max(ans, tr[x]);  
        return ans;  
    }  
};  
  
void work(){  
    int n;  
    cin >> n;  
    vector<int> a(n + 1), b;  
    vector<ll> w(n + 1);  
    for(int i = 1; i <= n; ++i){  
        cin >> a[i] >> w[i];  
        b.push_back(a[i]);  
    }  
    sort(all(b));  
    b.erase(unique(all(b)), b.end());  
    BIT bit(sz(b));  
    ll ans = 0;  
    for(int i = 1; i <= n; ++i){  
        int x = lower_bound(all(b), a[i]) - b.begin() + 1;  
        ll cur = max(0ll, bit.query(x - 1)) + w[i];  
        bit.update(x, cur);  
    }  
}
```



```
    ans = max(ans, cur);  
}  
cout << ans << '\n';  
}
```